

Objetivo

Criar um formulário HTML com validação de senha, utilizando FastAPI como backend. O usuário preenche a senha no navegador e a aplicação verifica se a senha está correta, exibindo uma mensagem de sucesso ou erro.

O passo a passo abaixo foi testado em um ambiente visual code, porém pode ser replicado em outros ambientes. Siga o passo a passo.

Observacao: Ao instalar o python, pode ser que a palavra para criar no terminal seja python ou python3

Etapa 1: Instalar o Python

1. Acesse o site oficial do Python: <https://www.python.org/downloads>
2. Baixe e instale a versão mais recente do Python 3.x para o seu sistema operacional.
3. Durante a instalação, marque a opção **"Add Python to PATH"**.
4. Para verificar se a instalação foi feita corretamente, abra o terminal e digite:

```
python --version
```

Se houver erro tente com python3, por exemplo:

```
python3 --version
```

Etapa 2: Criar a Estrutura do Projeto

1. Crie uma pasta chamada **formulario_fastapi**:

```
mkdir formulario_fastapi  
cd formulario_fastapi
```

ou crie diretamente sem precisa usar o script acima

2. Crie uma subpasta chamada **templates** (para armazenar arquivos HTML):
-

```
mkdir templates
```

ou crie diretamente sem precisa usar o script acima

Estrutura de arquivos e pastas

```
formulario_fastapi/  
├── main.py  
├── templates/  
│   └── index.html
```

Etapa 3: Criar Ambiente Virtual (opcional, mas recomendado)

Para criar a pasta no ambiente virtual, certifique-se que o terminal está apontando para a pasta raiz `formulario_fastapi`. Para detalhes procure por comando `bash` ou `prompt de comando`

```
python -m venv .venv  
source .venv/bin/activate    # Comando para ser executado se for  
Linux/macOS  
.venv\Scripts\activate      # Comando para ser executado se for Windows
```

pode ocorrer de a instalação ser a palavra `python3`, ficaria assim

```
python3 -m venv .venv  
source .venv/bin/activate    # Comando para ser executado se for  
Linux/macOS  
.venv\Scripts\activate      # Comando para ser executado se for Windows
```

Estrutura final de arquivos e pastas

```
formulario_fastapi/  
├── main.py  
├── templates/  
│   └── index.html  
└── .venv/
```

Etapa 4: Instalar Dependências

No terminal, execute:

```
pip install fastapi uvicorn jinja2 python-multipart
```

São bibliotecas necessárias para executar o processo

Explicação de cada biblioteca:

- **fastapi**: Framework para criar APIs modernas e rápidas com Python.
- **uvicorn**: Servidor ASGI que executa a aplicação FastAPI.
- **jinja2**: Template engine para renderizar HTML dinâmico.
- **python-multipart**: Necessário para processar dados de formulários HTML (`Form(...)`).

Etapa 5: Criar o Arquivo do Backend (main.py)

Crie um arquivo chamado `main.py` com o seguinte conteúdo:

```
from fastapi import FastAPI, Form, Request
from fastapi.responses import HTMLResponse, JSONResponse
from fastapi.templating import Jinja2Templates

app = FastAPI()

templates = Jinja2Templates(directory="templates")

@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse("index.html", {"request":
request})

@app.post("/validar", response_class=JSONResponse)
async def validar_senha(senha: str = Form(...)):
    senha_correta = "1234seguro"
    return {"valido": senha == senha_correta}
```

Etapa 6: Criar o Arquivo HTML (templates/index.html)

Crie o arquivo `templates/index.html` com o seguinte conteúdo:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Validação de Senha</title>
  <style>
    body { font-family: Arial; padding: 2rem; background: #f2f2f2; }
    form { background: white; padding: 2rem; max-width: 400px; margin:
```

```

auto; border-radius: 8px; }
  label, input { display: block; margin-bottom: 1rem; width: 100%; }
  #mensagem { font-weight: bold; margin-top: 1rem; }
  .erro { color: red; }
  .sucesso { color: green; }
</style>
</head>
<body>
  <form id="formLogin">
    <h2>Login</h2>
    <label for="senha">Senha:</label>
    <input type="password" id="senha" name="senha" required>
    <button type="submit">Entrar</button>
    <p id="mensagem"></p>
  </form>

  <script>
    const form = document.getElementById("formLogin");
    const mensagem = document.getElementById("mensagem");

    form.addEventListener("submit", async function(e) {
      e.preventDefault();
      const formData = new FormData(form);
      const resposta = await fetch("/validar", {
        method: "POST",
        body: formData
      });
      const resultado = await resposta.json();
      if (resultado.valido) {
        mensagem.textContent = "\u2705 Senha correta. Acesso
permitido!";
        mensagem.className = "sucesso";
      } else {
        mensagem.textContent = "\u274C Senha incorreta. Tente
novamente.";
        mensagem.className = "erro";
      }
    });
  </script>
</body>
</html>

```

Etapa 7: Rodar o Projeto

No terminal, dentro da pasta do projeto, execute:

```
uvicorn main:app --reload
```

Abra o navegador e acesse: <http://127.0.0.1:8000>

Teste o formulário:

- Senha correta: 1234seguro
- Senha errada: qualquer outra



Resultado Esperado

O navegador exibirá uma mensagem em verde caso a senha esteja correta, ou uma mensagem em vermelho se estiver incorreta, tudo sem recarregar a página.
